

Monte Carlo Simulation Based Estimations: Case from a Global Outsourcing Company

J. Dasgupta, Member IEEE, Member IEEE Computer Society
Patni Computer Systems Limited
Airoli, Navi Mumbai 400 708, India
jyotidasgupta@yahoo.co.in

G. Sahoo
Department of Information Technology
Birla Institute of Technology
Mesra, Ranchi 835 215, India
drgsahoo@yahoo.com

R. P. Mohanty
ITM Group of Institutions
B.S.E.L. Tech Park, 704-712,
Sector 30 A, Vashi, Navi Mumbai 400 705, India
rpmohanty@gmail.com

Abstract

Effort estimation of various software project life cycle phases have been traditionally performed using deterministic methods. Deterministic methods fail to account for the variability inherent in the software engineering processes deployed in executing the project. We have developed stochastic notations and models that can represent these process variations. We have also constructed a Monte Carlo simulation tool that works using the stochastic model we have developed for software development projects. This tool has been tested in a mid-sized software services outsourcing (SSO) organization. The tool and results are described and reported in this paper.

Introduction

Software project risk management focuses on assessing various risks that need to be managed during the entire life cycle of the project from proposal to retirement. In the absence of effective risk assessment and mitigation, software projects may not allow the organization to meet its business objectives. In the context of software services outsourcing (SSO) industry, software project risk management assumes higher significance as competitiveness and continued success of a SSO organization depends on estimating the risks associated with hundreds and often thousands of software projects of various types executing concurrently at any time. [1]

Although SSO must manage both financial and operational risks, operational risk management is considered part of project management processes in all leading global and industry standards. Various techniques and indices for assessing and managing software project risks, and the risk factors that influence the risks, have been devised and reported by Dasgupta and Mohanty. [2] Additionally, risk representation through innovative risk radars have also been proposed and tested in a mid sized SSO organization operating globally. [3]

These above operational risk management methods have been further extended in this paper through development of stochastic models of the underlying software engineering processes, and use of Monte Carlo simulation methods. Advent of powerful computers and simulation software have made it possible for software project managers to use Monte Carlo simulation to predict likely range within which a project performance i.e. effort, schedule or quality may be expected to lie.

Research methodology used was:

- Literature review on use of Monte Carlo simulation in risk management
- Develop a mathematical model to estimate effort spent in software development projects. Notations to depict stochastic variables and mathematical operations have been developed and used in the model
- Collection of stochastic project process performance data from 70 software development projects
- Perform Monte Carlo simulation of the input and output stochastic process performance variables using a tool developed using a commercially available platform @Risk from Palisade software
- Analysis and reporting of results

Brief Review of Monte Carlo Simulation Techniques

Invention of Monte Carlo simulation techniques is usually credited to Stanislaw Ulam, a Polish mathematician, while working at the Los Alamos laboratory in the USA, where numerical simulations were first used by atom bomb scientists. These methods were further developed by John Von Neumann and others. As these simulations required an enormous amount of computation, their uses were restricted only to few areas until recently when advances in computation techniques and power made it useful in many industries, most notably finance. At present, Monte Carlo simulation is the main stay of Value at Risk computation, an indispensable part of managing financial risk. [4] With help from modern software packages, Monte Carlo experiments

can be very powerful tool in supporting decision making process under uncertainty. [5]

Review of literature reveals widespread use of Monte Carlo simulations in managing risks such as in construction projects [6] managing investments [7] assessing and managing operational risk for banks [8] planning a corpus for retirement [9] financial risk assessment of BOT projects [10] computing Value at Risk [11] assessing risk in building railway tunnels [12] risk based estimation models [13] or estimating schedule risks in construction projects. [14]

Use of Monte Carlo simulation in software projects is relatively much less. One of the earliest uses is observed in assessing sensitivity in the COCOMO II model for software cost estimation. [15]. It has also been used for assessing staffing needs in software maintenance projects. [16] Lately, the Software Engineering Institute (www.sei.cmu.edu) has been advocating the use of various statistical techniques including Monte Carlo simulation for quantitative project management However, software organizations have only recently started experimenting with these methods.

We have developed a model for using Monte Carlo simulation techniques to estimate software development project effort estimation risks. This model has been implemented in a mid-sized software services company, and results and significant findings have been reported in this paper. This work is expected to benefit other software companies through more useful ‘confidence’ based effort estimations for software development projects.

Stochastic Model for Software Development Project Risk Estimation

Monte Carlo simulations replace deterministic values in mathematical equations with probabilistic values. Multiple solutions of the equation are computed by randomly sampling each probability distribution of every value and an array of statistics are generated. A generalized form for mathematical equation can be

$$Y = f(X_1, X_2, \dots X_k) \dots\dots\dots (1)$$

Mathematical equations of above type are often used in building mathematical models. Values of X_n are independent variables that are estimated based on historic values, assumptions or working hypotheses. These types of models are deterministic as range of values of independent variable are not considered. [17]

This problem can be addressed by employing best – worst – likely scenarios, or by performing what – if analysis using a range of input values for X_k .

Another method is to replace the deterministic values by stochastic variables by using probability distribution functions (PDF). The probability distributions are selected to represent the underlying values and variability of the input independent variables.

To differentiate between deterministic and stochastic values of Y and X_k we would use Greek letters Ψ (pronounced PSI) to denote output or dependent stochastic variable Y with its PDF and Φ (pronounced PHI) to denote input or independent stochastic variable X_k with PDF.

$$\Psi = Y \text{ (PDF)} \dots\dots\dots (2)$$

$$\Phi = X_k \text{ (PDF)} \dots\dots\dots (3)$$

So we modify Equation 1 to below.

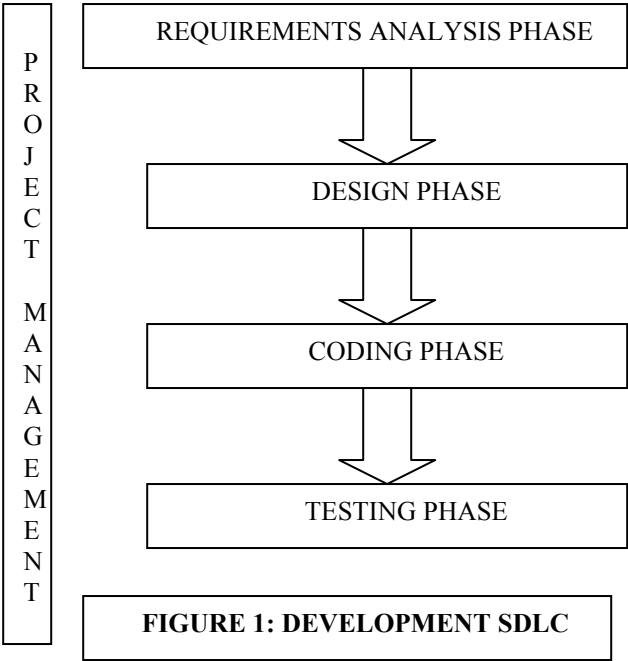
$$\Psi = f(\Phi_1, \Phi_2, \dots \Phi_k) \dots\dots\dots (4)$$

Equation 4 would be further developed for software development project effort estimation and used for model development in this paper.

Software Project Life Cycle Phases

There are many types of projects such as development, reengineering, migration, maintenance, verification and validation, production support etc. Dasgupta and Mohanty have described these project types in detail. Each of these project types follow different life cycles. [2]

Software development life cycle (SDLC) for ‘development’ type of projects is shown in Figure 1.



This figure shows the main phases of a software development project. Known as the waterfall model, the diagram depicts the phases being completely independent, with project management activities spanning across all the phases.

The project manager is always the first person to join a project, and often the last one to leave. Other phases start and end, as shown in Figure 1. In reality there are overlaps and iterations among the phases.

Within each of these phases, there are sub-phases. The phases and sub-phases are listed below.

1 Requirements Analysis Phase: In this phase, requirements of the system under development are captured and documented using formal and structured techniques.

1.1 Requirement capture: In this sub-phase, software project requirement analysts, which is a specialist function,

interact with the clients and end users of the system to fully understand and document the scope, functional and non-functional (for example performance, regulatory compliance etc) requirements of the software to be built

1.2 Requirement review: the documented requirements are reviewed by peers and users to ensure that requirements have been well understood and documented

1.3 Requirement rework: correcting defects found during review or later in the project life cycle are termed rework

2. Design Phase: High level and detailed design of the software is produced in the phase. This phases requires design specialists, who create the over all architecture of the software, as well as the detailed flowchart of the software. This phase often produces pseudo-codes, which are very close to actual program code but written using structured English. The sub-phases are:

2.1 Design Creation: Creation of the design documents as detailed above

2.2 Design review: performed by peer groups using formal review techniques, this sub-phase is used to detect errors in design

2.3. Design rework: corrections that need to be performed in the design as a result of the reviews, or later in the project life cycle

3 Coding Phase: Also known as construction phase, program codes are written in this phase. Programs may be written using a variety of languages and development aides.

3.1 Coding: Writing several programs which together form the complete software system.

3.2 Code review: Performed by peers, this sub-phase ensures that the programs are written using good programming practices, and are error free.

3.3 Code rework: Effort spent in correcting errors or defects found in the programs.

4 Testing: In this phase, software programs constructed are tested thoroughly. This is done using various methodologies and techniques. Key sub-phases are described below.

4.1 Unit Test cases: The basic building blocks of the software system under development are termed units. These are program units that can be tested individually for correct functioning. These tests are conducted by testers who can be other programmers or specialized testers.

4.1.1 Unit Test cases creation: In this sub-phase, testers design test scenarios to test that the units are functioning as per design.

4.1.2 Review: Unit Test cases are reviewed for correctness by peers

4.1.3 Rework: Errors found in test cases are corrected in this sub-phase

4.2 Unit Test cases execution: The test cases are executed by running the program units along with the test scenario inputs, and validating that the outputs are as per design.

5 System and Integration: All units are integrated and tested in this phase. Rework required as a result of tests in this phase are captured under this sub-phase

6 Project Management: This phase spans across and runs concurrently with all above phases and sub-phases.

Project Process Performance

Organizations design and implement software engineering processes in order to manage and execute above phases and sub-phases consistently. Consistencies in the way these processes are scheduled and executed ensure predictability in project effort, schedule and work product quality.

Organization to organization variation in the process design and therefore process performance may be expected. Even within the same organization the process performance are observed to vary.

Observed variation may be caused due to epistemic or ontological reasons. [18]

Epistemic variations are caused by imperfections in our knowledge, which may be reduced by more research and empirical efforts. Ontological variations are caused by inherent variability in human, natural or technological systems. Ontological variations can not be reduced by additional research.

Reduction in process variability is desirable in engineering activity, including software engineering. Eliminating variability whether epistemic or ontological is a near impossibility, and hence should be taken into account while planning project performance.

Project planning therefore must account for the inherent variability by employing stochastic instead of deterministic methods. As all processes would display different variability, accounting for stochastic behavior using Equation 4 is required.

Stochastic Notations for Project Processes

Let us assume that efforts spent in the various sub-phases above are as under.

$E_{\text{PHASE OR SUBPHASE}}$ = Effort spent in phase pr sub-phase of a project

Φ_R = PDF of requirement capture effort in sample projects

Φ_{RR} = PDF of requirement review effort in sample projects

Φ_{RRW} = PDF of requirement rework effort in sample projects

Φ_D = PDF of design effort in sample projects

Φ_{DR} = PDF of design review effort in sample projects

Φ_{DRW} = PDF of design rework in sample projects

Φ_C = PDF of coding effort in sample projects

Φ_{CR} = PDF of code review in sample projects

Φ_{CRW} = PDF of coding rework in sample projects

Φ_U = PDF of Unit Test cases creation effort in sample projects

Φ_{UR} = PDF of Unit Test cases review effort in sample projects

Φ_{URW} = PDF of Unit Test cases rework effort in sample projects

Φ_{UE} = PDF of Unit Test cases execution effort in sample projects

Φ_S = PDF of System and Integration effort in sample projects

Φ_P = PDF of Project Management effort in sample projects

Stochastic Model for Software Development Projects

During Monte Carlo simulation, random values of each of the above stochastic variables would be generated in each pass. Let us say that there are a total of n passes and in each pass is denoted by k, with k between 1 and n. Project phase efforts in the kth pass then are not stochastic but deterministic as decided by the random number generator. So the deterministic equations, assuming E denotes the effort would be

$$E_{\text{REQUIREMENT } k} = E_{Rk} + E_{RRk} + E_{RRWk} \dots\dots\dots (5)$$

$$E_{\text{DESIGN } k} = E_{Dk} + E_{DRk} + E_{DRWk} \dots\dots\dots (6)$$

$$E_{\text{CODING } k} = E_{Ck} + E_{CRk} + E_{CRWk} \dots\dots\dots (7)$$

$$E_{\text{UNIT TEST } k} = E_{Uk} + E_{URk} + E_{URWk} + E_{Ek} \dots\dots\dots (8)$$

$$E_{\text{SIT } k} = E_{Sk} \dots\dots\dots (9)$$

$$E_{\text{PM } k} = E_{Pk} \dots\dots\dots (10)$$

Let us next denote the stochastic version of the above equations for all n passes as below.

$$\Psi_{\text{REQUIREMENT } k (k=1 \rightarrow n)} = f(\Phi_{Rk} + \Phi_{RRk} + \Phi_{RRWk}) \dots\dots (11)$$

$$\Psi_{\text{DESIGN } k (k=1 \rightarrow n)} = f(\Phi_{Dk} + \Phi_{DRk} + \Phi_{DRWk}) \dots\dots (12)$$

$$\Psi_{\text{CODING } k (k=1 \rightarrow n)} = f(\Phi_{Ck} + \Phi_{CRk} + \Phi_{CRWk}) \dots\dots (13)$$

$$\Psi_{\text{UNIT TEST } k (k=1 \rightarrow n)} = f(\Phi_{Uk} + \Phi_{URk} + \Phi_{URWk} + \Phi_{Ek}) \dots\dots (14)$$

$$\Psi_{\text{SIT } k (k=1 \rightarrow n)} = f(\Phi_{Sk}) \dots\dots\dots (15)$$

$$\Psi_{\text{PM } k (k=1 \rightarrow n)} = f(\Phi_{Pk}) \dots\dots\dots (16)$$

Project Process Performance Models Simulation

Having laid the notations and model for both the deterministic values of the effort generated by a random number generator, as well as stochastic PDF of both input and output variables in the model, we next proceed to run the model in Monte Carlo simulation software. There are several good tools available now that help in building Monte Carlo simulation models. For our experiments, we chose @Risk simulation software from Palisade (www.palisade.com). The steps followed are:

For the project for which we wish to estimate various phase efforts, we input the required data in a Microsoft Excel 'dialogue screen' set up as shown in Table 1.

In this screen the Project Manager (PM) is asked to enter data in the cells colored yellow. In this example, the PM first inputs data concerning the projects size expressed in 'Function Points', and the number of requirement analysts, designers, programmers and testers the PM had planned to deploy on the project. The PM also adds the number of management personnel including the PM. The PM is asked next to feed the confidence level the PM wants to operate at.

For this example we have chosen 90% - but the PM may choose higher (say 95%) or lower (say 80%) depending upon the criticality of the project and availability of personnel.

Table 1: PM Dialogue Window

Table 1: Project Manager Dialogue Window				
Project parameters	Desired confidence level	Predicted Effort at the confidence level (Hrs)	Predicted Duration working days@ 9 Hrs/ day)	
Estimated project size in Function Points	500			
Number of requirement analysts	3	90	368	13.63
Number of designers	5	90	1025	22.78
Number of programmers	10	90	3140	34.89
Number of testers	10	90	2000	22.22
SIT Phase staffing	20	90	1970	10.94
PM	1	90	1245	138.33

Function Point as the Size Unit of Development Projects

As project sizes differ, all historical data must be normalized by dividing by the size of the project. In this case, 'Function Points' – a globally recognized standard for software size has been used. Details of the size computation are available from www.ifpug.org.

The International Function Point Users' Group (IFPUG) is a non-profit, member governed organization. IFPUG defines methods to compute Function Points to measure size of application development and maintenance projects through the use of Function Point Analysis and measurement techniques. IFPUG endorses FPA as its standard methodology for software sizing. In support of this, IFPUG maintains the Function Point Counting Practices Manual, the recognized industry standard for FPA.

Performing Monte Carlo Simulation

This paper will not get into the specifics of using Monte Carlo simulation software as step by step instructions are easily available from tool providers such as @Risk from Palisade, Crystal ball from Oracle, Simulacion, Simular, etc. All these tools work with Microsoft Excel, and are quite simple to set up and use.

In general – the steps are:

- Detect frequency distribution of the input variables (X) in the input cells of the spreadsheet being used
- Define an output cell that contains the process model i.e. Equations 5 to 10, which when used in the tool would behave like Equations 11 to 16, and create distribution functions instead of single numbers
- Perform sensitivity analysis to understand which sub-phases or phases contribute higher variability. This allows the Project Manager to do a root cause analysis and reduce variations in those processes

We collected these data in 70 development projects executed in 2010 at a mid sized SSO firm. The resulting effort values at the desired confidence level are shown in Table 2. Inputting those values in the predicted effort cells in the dialogue screen in Figure 1 gives Phase duration calculated assuming work days of X hours/ work day. PM can now proceed to perform 'what if' analysis through changing the

staffing levels or work hours of various phases or desired confidence levels.

Table 2: Phase wise Effort Predictions

Table 2: Phase-wise effort prediction			
		Person Hours/ FP	Value at desired confidence level
REQUIREMENT PHASE	Requirement effort	0.421156242	
	Requirement review	0.076728796	
	Requirement rework	0.052594	
	TOTAL REQUIREMENT PHASE	0.550479038	0.736
DESIGN PHASE	Design effort	0.613361425	
	Design review	0.598106971	
	Design Rework	0.169461362	
	TOTAL DESIGN PHASE	1.380929778	2.05
CODING PHASE	Coding	2.799940974	
	Code review	0.386121248	
	Coding rework	0.22286945	
	TOTAL CODING PHASE	3.408931673	6.28
TESTING PHASE	Unit Test creation	0.40797142	
	Unit Test Execution	2.13668	
	Unit Test Review	1.397039277	
	Unit Test rework	0.037724633	
TOTAL UNIT TEST PHASE		3.97941533	4
	System & Integration Test related rework	1.352907	3.94
PROJECT MANAGEMENT	Project management	1.083023128	2.49

Detecting PDF of the Past Projects Sub-Phases

The underlying engine of the tool is the effort PDF of every sub-phase collected from the 70 projects. These PDF are computed by feeding the past project data and detecting distribution of the sub-phase wise data. Sub-phase wise PDF for these projects are shown in Table 3.

Table 3: Model inputs

Table 3: Model inputs					
Name	Cell	Graph	Function	Min	Mean
Requirement Effort	D4		RiskBetaGeneral(0.59162,0.47455,0.2,0.59855,RiskName("Requirement Effort"))	0.2	0.4211562
Requirement Review	D5		RiskWeibull(2.7156,0.035265,RiskShft(0.045362),RiskName("Requirement Review"))	0.045362	0.0767288
Requirement Rework	D6		RiskLogistic(0.052594,0.010467,RiskName("Requirement Rework"))	-∞	0.052594
Design	D8		RiskBetaGeneral(0.71174,0.66613,0.20993,0.99094,RiskName("Design"))	0.20993	0.6133614
Design Review	D9		RiskExValue(0.49482,0.17894,RiskName("Design Review"))	-∞	0.598107
Design Rework Effort	D10		RiskLogistic(0.0039593,0.050113,1.3369,RiskName("Design Rework Effort"))	0.0039593	0.1694614
Coding	D12		RiskBetaGeneral(0.76971,2.3713,1.138,7.92,RiskName("Coding"))	1.138	2.799941
Code Review	D13		RiskBetaGeneral(0.92067,1.5336,0.095483,0.87025,RiskName("Code Review"))	0.095483	0.3861212
Coding rework	D14		RiskExpn(0.22327,RiskShft(-0.00040055),RiskName("Coding rework"))	-0.00040055	0.2228695
Unit Test Creation	D16		RiskExValue(0.31336,0.16391,RiskName("Unit Test Creation"))	-∞	0.4079714
Unit Test Execution	D17		RiskUniform(0.92716,3.3462,RiskName("Unit Test Execution"))	0.92716	2.13668
Unit Test Case Review	D18		RiskLogistic(0.0019893,0.086009,1.062,RiskName("Unit Test Case Review"))	0.0019893	1.397039
Unit Test Case rework	D19		RiskLogistic(0.0019199,0.024977,2.2179,RiskName("Unit Test Case rework"))	0.0019199	0.03772463
SIT Phase rework	D21		RiskExpn(1.2972,RiskShft(0.055707),RiskName("SIT Phase rework"))	0.055707	1.352907
Project Management and other tasks	D22		RiskWeibull(1.2836,0.9661,RiskShft(0.16943),RiskName("Project Management and other tasks"))	0.16943	1.083023

Creating PDF of the Phase Wise Efforts

This is performed by the tool automatically. Simulation parameters like number of iterations and other factors can be set by the PM. Phase wise effort PDF is given in table 4.

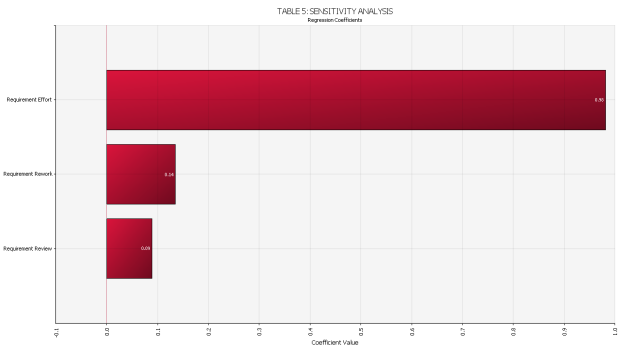
Table 4: Simulation Model Outputs

Table 4: Simulated Model Outputs								
Name	Cell	Graph	Min	Mean	Max	5%	95%	Errors
TOTAL REQUIREMENT PHASE / Person Hours) FP	D7		0.2764538	0.5504997	0.7914815	0.3288255	0.737477	0
TOTAL DESIGN PHASE / Person Hours) FP	D11		0.4446826	1.354334	9.567351	0.7486624	2.083023	0
TOTAL CODING PHASE / Person Hours) FP	D15		1.333254	3.408701	8.07866	1.615568	6.394945	0
TOTAL UNIT TEST PHASE / Person Hours) FP	D20		1.204134	3.286001	305.0574	1.592192	4.461616	0
System & Integration Test related rework / Person Hours) FP	D21		0.05679328	1.352931	10.16856	0.1220978	3.919809	0
Project management and other tasks / Person Hours) FP	D22		0.1709672	1.082658	4.707087	0.285327	2.485635	0

Sensitivity Analysis

PM can also perform sensitivity analysis of the simulated results. Sub-phases with higher sensitivity display higher variances. PM can analyze the data, try to understand which variations are caused by epistemic or ontological reasons, and then try to reduce the variation by initiating appropriate actions that allows the project to be completed with lesser efforts than what could be achieved in the past in these sample projects. Sensitivity analysis is displayed in Table 5. Factors with higher sensitivity are depicted as the longer bars.

Table 5: Sensitivity Analysis



Significant Findings

Monte Carlo simulations have been used for decision making under uncertainty for past several decades in various fields including project management. As the simulations are based on detecting the probability distribution functions of the independent input variables, and in simulating mathematically thousands of possible scenarios to derive PDF of the dependent output variable, the underlying assumption is that the processes that are used to execute the project have the ability to perform consistently. Without consistent performance, both epistemic and ontological variations may be hard to control.

Software engineering is a relatively new branch of engineering. Consistency in the software engineering processes have therefore been achieved relatively late compared to other branched of engineering.

With consistency in software engineering processes, and thus better understanding of the variations inherent in these

processes, it is possible to predict the PDF of the dependent variables of software engineering processes. This allows better project management compared to the older methods where deterministic values were sought, ignoring the variations in the independent variables.

In this paper, we have achieved the following:

- Proposed stochastic notations that allow easier modeling of software engineering process performance models
- Using the notations developed project process models for software development projects
- Obtained stochastic project performance data from 70 development projects
- Used the stochastic process model developed in this paper to drive simulation of output variables by using a tool developed on commercially available software @Risk.
- Explained how PMs have and can use these concepts and models to easily create and replicate similar tools and simulation models

Limitations and further research

In this paper we have developed notations and models for software development projects only. Models should be developed for other project types as well. [2] Software engineering process performance varies from organization to organization, as well as with differing technologies, tools usage, personnel skills etc. We claim that the methods and model developed in this paper are easily replicable across industries an project processes.

Concluding Remarks

As described earlier, advent of modern computer based Monte Carlo simulation packages such as Crystal Ball (www.oracle.com), Simulacion (www.ucema.edu.ar/~jvarela/index_eng.htm) or @Risk (www.palisade.com) make it possible for software project managers to include variability into their project plans. Until recently such simulations demanded formidable mathematical computations. As we demonstrate in this paper, stochastic models can be developed for all types of projects in every industry using the notations and methods developed in this paper.

Acknowledgment

The authors would also like to sincerely thank the software organization where these experiments were conducted.

References

1. Dasgupta, J. and Mohanty, R. P., "Towards evaluating the risks of software services outsourcing industry", *Vilakshan: XIMB Journal of Management*, Vol. 6, No. 2 (2009), pp 29-48
2. Dasgupta, J. and Mohanty, R. P., "Estimating operational risk indices for software services outsourcing industry: a case", *Int. J. Services and Standards*, Vol. 6, No. 1 (2010), pp 43 – 61
3. Dasgupta, J. and Mohanty, R. P., "Towards formulation of software project risk radars", *Int. J. Business Information Systems*, Vol. 6, No. 2 (2010), pp 151 – 178
4. Jorion, P. Value at Risk: The New Benchmark for Managing Financial Risk, Tata McGraw Hill, (New Delhi, 2009)
5. Gujarati, D. N., Sangeetha, Basic Econometrics, Tata McGraw-Hill (New Delhi, 2008)
6. Khedr, M. K., "Project risk management using Monte Carlo simulation", *AACE International Transactions*, (2006) pp 02.1 – 0.2.10
7. Demir, H. and Bostanci, B., "Decision-support analysis for risk management", *African Journal of Business Management*, Vol. 4(8) (2010), pp 1586 – 1604
8. Ergashev, B., "Estimating the lognormal-gamma model of operational risk using the Markov chain Monte Carlo method", *The Journal of Operational Risk* (2009), pp 35 – 57
9. Tezel, A., McManus, G., Sharma, R. , "Reaching a retirement wealth target: A new life-cycle asset allocation strategy", *Journal of Financial Service Professionals*, July 2010, pp 62 – 68
10. Sung, C. and Kuo, C., "Financial risk analysis of BOT projects – the case of ICCTPKH", *International Research Journal of Finance and Economics*, Issue 42 (2010), pp 109 – 128
11. Suhobokov, A., Application of Monte Carlo simulation methods in risk management, *Journal of Business Economics and Management*, Vol. VII, No. 3 (2007)., pp 165 -168
12. Zhi-sheng, X., Zhi-jun, H., Ping, S., "The application of risk management to investment of the railway tunnel project", *The first International Conference on Information Science and Engineering, IEEE Computer Society* (2009), pp 4547 – 4550
13. Cretu, O. and Berends, T., Risk-based estimate self-modeling, 2009 AACI International Transactions(2009), pp RISK.SO2.1 – SO2.25
14. McCabe, B., "Monte Carlo simulation for schedule risks", *Proceedings of the 2003 Winter Simulation Conference* (2003), pp 1561 – 1565
15. Musilek, P., Pedrycz, W., Sun, N., Succi, G., "On the sensitivity of the COCOMO II Software Cost Estimation model", *Proceedings of the eighth symposium on Software Metrics, IEEE Computer Societ* (2002)y, pp 1 - 8
16. Antoniol, G., Cimitile, A., Di Lucca, G. A., Di Penta, M., "Assessing staffing needs for a software maintenance project through queuing simulation", *IEEE Transactions on Software Engineering*, Vol. 30, No. 1 (2004), pp 43 – 58
17. Evans, G. E., Jones, B., "The application of Monte Carlo simulation in Finance, Economics and Operations Management", *Proceedings of the 2009 World Congress on Computer Science and Information Engineering, IEEE Computer Society* (2009), pp 379 – 383
18. Salling, K. B., Leleur, S., "Assessment of transport infrastructure projects by the use of Monte Carlo simulation: The CBA – DK model", *Proceedings of the 2006 Winter Simulation Conference* (2006), IEEE, pp. 1537 – 1544